Homework assignment 2

20254251 Computational Imaging, Fall 2025/2026
Weizmann Institute of Science

Due Friday, Jan. 2, at 11:59pm ET

The purpose of this assignment is to explore high dynamic range (HDR) imaging, color calibration, and tonemapping. As we discussed in class, HDR imaging can be used to create floating-point precision images that linearly map to scene radiance values. Color calibration ensures that the colors you see in the image match some groundtruth RGB values. Tonemapping algorithms compress the dynamic range of HDR images to an 8-bit range, so that they can be shown on a display. To get full credit, you will need to apply these steps to both an exposure stack provided by us, ~~and one that you capture yourselves~~ (you're welcome).

Throughout the assignment, we refer to a number of key papers that we mentioned in class. While the assignment and class slides describe most of the steps you need to perform, we highly recommend that you read the associated papers.

Towards the end of this document, you will find a "Deliverables" section describing what to submit. Throughout the writeup, we also mark in red questions you should answer in your report. Lastly, there is a "Hints and Information" section at the end of this document that is likely to help. We strongly recommend that you read that section in full before you start to work on the assignment. The Python packages required for this assignment are `numpy`, `skimage`, `matplotlib`, and `cv2` (OpenCV, to read and write HDR files), and you can use the functions provided in the `./src/cp_hw2.py` file of the homework ZIP archive.

1. HDR imaging (50 points)

For this and the following two parts (color correction, and tonemapping), you will use an exposure stack provided we captured in the `./data/image_stack` directory of the homework ZIP archive. The provided directory contains a set of `.JPG` files, please also download the original `.NEF` files from here and add them to the folder. Figure 1 shows two exposures, as well as a (tonemapped) HDR composite.

While not particularly beautiful, the scene has a number of features that make it a good example for HDR: First, there are two areas with very different illumination and dynamic range that no single exposure can simultaneously capture correctly. Second, both areas include colorful items (Toy Story poster in the background of the bright area, Plus-Plus pieces, SIGGRAPH mugs, and book covers in the dark area) that you can use to evaluate the color rendition of your results. Third, the in-focus area has high-detail features (lettering on the book covers and lens/camera markings) that you can use to evaluate the resolution of your results. Finally, the scene includes a color checker than you can use for color calibration.

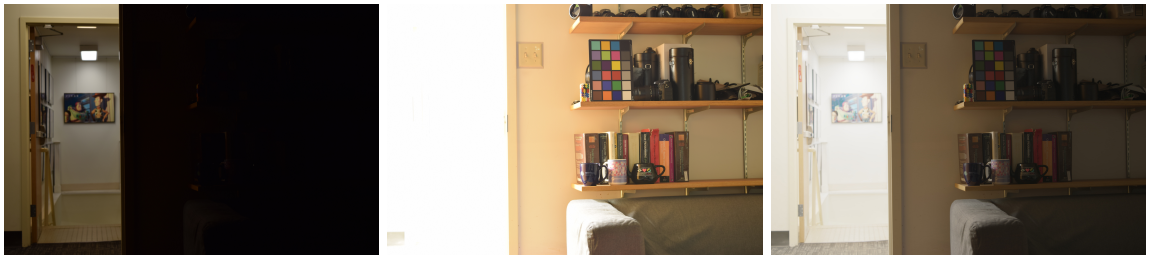

Figure 1: From left to right: Two LDR exposures, and an HDR composite after photographic tonemapping.

Your data folder contains two sets of images: RAW (`.NEF`) and rendered (`.JPG`). As we discussed in class, the procedure for merging many low dynamic range (LDR) exposures into an HDR image is different for RAW and rendered images. To appreciate the difference, in this assignment you will create HDR images from both sets of images.

For reference, we captured both exposure stacks with fixed aperture and ISO, and with shutter speeds equal to $\frac{1}{2048} \cdot 2^{k-1}$ [sec], where $k \in \{1, \ldots, 16\}$ is the index in an image's file name.

Develop RAW images (10 points). Use `dcraw` to convert the RAW `.NEF` images into *linear* 16-bit `.TIFF` images. For this, you should direct `dcraw` to do white balancing using the camera's profile, do demosaicing using high-quality interpolation, and use sRGB as the output color space. Read through `dcraw`'s documentation to work out what the correct flags for this conversion are. Report the flags you use.

Merge exposure stack into HDR image (40 points). Now you have a set of k LDR linear images having corresponding different exposures t^k that you can merge into a single HDR image. We first introduce some notation. We use $I^k_{ij,\text{LDR}}$ to refer to the intensity value of pixel $\{i, j\}$ of the k-th *original* LDR input image, read directly from the `.TIFF` file. We use $I^k_{ij,\text{lin}}$ to refer to the intensity of pixel $\{i, j\}$ of the k-th *linear* LDR input image; for our `.TIFF` images, $I^k_{ij,\text{lin}}$ is the same as $I^k_{ij,\text{LDR}}$. Later, when working with the `.JPG` image, these two terms will differ. Additionally, from this point on we assume that $I^k_{ij,\text{LDR}} \in [0, 1]$. Therefore, you need to normalize the original LDR input images to the $[0, 1]$ range, which you can do by dividing with $2^{16} - 1$ when using `.TIFF` files.

With this notation at hand, we form the HDR image as:

$$I_{ij,\text{HDR}} = \frac{\sum_k w\big(I^k_{ij,\text{LDR}}\big) I^k_{ij,\text{lin}}/t^k}{\sum_k w\Big(I^k_{ij,\text{LDR}}\Big)}. \tag{1}$$

As explained in class, the weights w in Equations (1) place more emphasis on well-exposed pixels, and less emphasis on under-exposed or over-exposed ones. See below about what weights to use. Implement the HDR merging and store the resulting HDR images as `.HDR` files, which is an open source high dynamic range file format. (See the provided function `writeHDR` in `./src/cp_hw2.py`). You can view the resulting files with the following viewer https://viewer.openhdr.org/.

Weighting schemes. There are many possible weighting scheme choices [3]. You will implement four:

$$w_{\text{uniform}}(z) = \begin{cases} 1, & \text{if } Z_{\min} \le z \le Z_{\max} \\ 0, & \text{otherwise} \end{cases},$$

$$w_{\text{tent}}(z) = \begin{cases} \min(z, 1-z), & \text{if } Z_{\min} \le z \le Z_{\max} \\ 0, & \text{otherwise} \end{cases},$$

$$w_{\text{Gaussian}}(z) = \begin{cases} \exp\Big(-4\frac{(z-0.5)^2}{0.5^2}\Big), & \text{if } Z_{\min} \le z \le Z_{\max} \\ 0, & \text{otherwise} \end{cases},$$

$$w_{\text{photon}}\big(z, t^k\big) = \begin{cases} t^k, & \text{if } Z_{\min} \le z \le Z_{\max} \\ 0, & \text{otherwise} \end{cases}. \tag{2}$$

All weighting schemes assume that intensity values $z \in [0, 1]$. You can experiment with different clipping values $Z_{\min}$ and $Z_{\max}$, but we recommend using $Z_{\min} = 0.02, Z_{\max} = 0.95$. Report the values you used. Unlike the other schemes, the weights w_{photon} also depend on the exposure under which a pixel was captured. Implement all the above weighting schemes, and use them to create HDR images (4 HDR images in total). Compare between the images and report any significant differences you may discover. Visualize the differences by picking three sections of the image to crop and show how it varies across different weighting schemes. These crops should make it clear what the differences across methods look like. Show all 12 crops (3 sections of the image x 4 weighting schemes) in your final report.

Make your pick. Select one out of the four HDR images you created. You can select, for example, the one that you find the most aesthetically pleasing. Make sure to comment on why you selected the image you did. Note that, as you have not yet tonemapped your HDR images, if you display them directly they will not look very nice; see "Hints and Information".

2. Color correction and white balancing (20 points)

Figure 2: Tonemapped HDR image without (left) and with (right) color correction.

For this part, you will use the HDR image you selected at the end of Part 1. As shown to the left of Figure 2, your tonemapped images will tend to have an orange cast in the dark parts of the room. This is because the very low light inside the room and the large contrast with the light outside the room are throwing the camera's automatic white balancing off. Additionally, even if the white balancing worked perfectly, we have not been very careful about the color space the various image composites reside in.

You **could** apply any of the automatic white balancing algorithms we discussed in Homework Assignment 1 to ameliorate the issue. But, given that the images include a color checker (Figure 3), it is possible to do better than that and perform accurate color correction.

Figure 3: Color checker, with white patch at the bottom-right corner.

In particular, the color checker is designed so that its patches have a specific set of RGB coordinates in the (linear) sRGB color space, when the color checker is viewed under a standard illuminant (so called "D65" illumination, roughly corresponding to daylight at noon). The function `read_colorchecker_gm`, which we provide in the `./src/cp_hw2.py` file of the homework ZIP archive, returns these ground-truth RGB coordinate values.

Then, to have your HDR image show color correctly, you can apply a linear transform on its three channels, so that the color checker's RGB coordinates in the image match the ground-truth coordinates as closely as possible. You can do this as follows.

1. For each color checker patch, crop a square that is fully contained within the patch. (Ask ChatGPT about the `cv2.setMouseCallback` function for interactively selecting points in the image.) Make sure

to store the coordinates of these cropped squares, so that you can re-use them. Use the resulting 24 crops to compute average RGB coordinates for each of the color checker's 24 patches.

2. Convert these computed RGB coordinates into *homogeneous* 4×1 coordinates, by appending a 1 as their fourth coordinate.

3. Solve a least-squares problem to compute an *affine transformation*, mapping the measured to the ground-truth homogeneous coordinates.

4. Apply the computed affine transform to your original RGB HDR image. Note that the transformed image may have some negative values, which you should clip to 0.

5. Finally, apply an additional white balancing transform (i.e., multiply each channel with a scalar), so that the RGB coordinates of the white patch (bottom-right patch in Figure 3) are equal to each other. This is analogous to the manual white balancing in Homework Assignment 1, where now we use the white patch as the white object in the scene.

Store the color corrected and white balanced HDR image in an `.HDR` file. You should now have two HDR images total: The one from Part 1 that has not been color-corrected, and the one you just created. Compare the color-corrected image with the original, and discuss which one you like the best. Additionally, create a "before and after" figure that compares the side by side results of the **cropped** color checker (as in Fig. 3).

3. Photographic tonemapping (10 points)

Now that you have a couple of HDR images, you need to tonemap them so that you can display them. For this part, you can use whichever of the two HDR images at the end of Part 2 you liked the best.

You will implement the tonemapping operator proposed by Reinhard et al. [4], which is a good tonemapping baseline. We describe how to do this below, but we strongly encourage you to read at least Sections 2 and 3 of this paper, which explain the rationale behind the specific form of this tonemapping operator, the effect of the various parameters, and its relationship to the zone system used when developing film.

Given pixel values $I_{ij,\mathrm{HDR}}$ of a linear HDR image, photographic tonemapping is performed as

$$I_{ij,\mathrm{TM}} = \frac{\tilde{I}_{ij,\mathrm{HDR}}\left(1 + \frac{\tilde{I}_{ij,\mathrm{HDR}}}{\tilde{I}^2_{\mathrm{white}}}\right)}{1 + \tilde{I}_{ij,\mathrm{HDR}}}, \tag{3}$$

where[1]

$$\tilde{I}_{\mathrm{white}} = B \cdot \max_{i,j}\left(\tilde{I}_{ij,\mathrm{HDR}}\right), \tag{4}$$

$$\tilde{I}_{ij,\mathrm{HDR}} = \frac{K}{I_{\mathrm{mean}}} I_{ij,\mathrm{HDR}}, \tag{5}$$

$$I_{\mathrm{mean}} = \exp\left(\frac{1}{N}\sum_{i,j}\log(I_{ij,\mathrm{HDR}} + \epsilon)\right). \tag{6}$$

The parameter K (*key*) in Eq. (5) shifts the image mean to a new values which determines how bright or dark the resulting tonemapped rendition is. The parameter B (*burn*) can be used to suppress the contrast of the result. It determines the maximum intensity value above which our resulting tonemapped image is allowed to clip bright pixels to 1. Finally, N is the number of pixels, and ϵ is a small constant to avoid the singularity of the logarithm function at 0.

Implement the photographic operator and apply it to your RGB HDR images in two ways: First, apply it by tonemapping all color channels simultaneously in the same way. Second, apply it only to the luminance

[1] Equation (6) is different from the corresponding Equation (1) in Reinhard et al. [4]. The version we give here is correct, and the version in the paper is incorrect.

channel Y. For the latter, you can use the provided function `lRGB2XYZ` to convert the HDR image from RGB to XYZ, and then convert it to xyY using the definition we discussed in class. While in xyY, tonemap the luminance Y while leaving the chromaticity channels x, y untouched. Then, invert the color transform to go back to RGB using the provided function `XYZ2lRGB`.

Experiment with different key and burn values. Some reasonable starting values for the parameters are $K = 0.15$ and $B = 0.95$, but to get good tonemaps you will need to explore different values. Plot representative tonemaps for both the RGB and luminance methods, and discuss your results. Make sure to mention which tonemap you like the most.

4. And now, for `.JPG` (20 points)

Now it is time to repeat the HDR creation process but using the corresponding `.JPG` images. Luckily, all the steps of the HDR pipeline you've implemented so far will remain the same. All you need to do is to linearize the `.JPG` images as described below.

Linearize rendered images (15 points). Unlike the RAW images, which are linear, the rendered images are non-linear. As we mentioned in class, before you can merge them into an HDR image, you first need to perform radiometric calibration to undo this non-linearity. You will do this using the method by Debevec and Malik [1]. We describe how this works below, but we strongly encourage you to read at least Section 2.1 of this paper, which explains the method.

An intensity $I_{ij}^k \in \{0, \dots, 255\}$ at pixel $\{i, j\}$ of image k relates to some unknown scene flux value L_{ij} as

$$I_{ij}^k = f\big(t^k L_{ij}\big), \tag{7}$$

where t^k is the (known) exposure of image k and f is the unknown non-linearity applied by the camera. If we knew f^{-1}, we could convert I_{ij}^k back to linear measurements.

Instead of f^{-1}, you will recover the function $g \equiv \log\big(f^{-1}\big)$ that maps pixel values I_{ij}^k to $g\big(I_{ij}^k\big) = \log(L_{ij}) + \log\big(t^k\big)$. This is motivated by the fact that the human visual systems responds to logarithmic, instead of linear, intensity. As the domain of g is the set of discrete intensity values $\{0, \dots, 255\}$, g is essentially just a 256-dimensional vector (i.e., an LUT).

Solving for these 256 values may seem impossible, because we know neither g nor L_{ij}. However, if the imaged scene remains static while capturing the exposure stack, we can take advantage of the fact that the value L_{ij} is constant across all LDR images. Then, we can recover g by solving the following least-squares optimization problem,

$$\min_{g, L_{ij}} \sum_{i,j} \sum_{k} \Big\{ w\big(I_{ij}^k/255\big) \big[g\big(I_{ij}^k\big) - \log(L_{ij}) - \log\big(t^k\big) \big] \Big\}^2 + \lambda \sum_{z=0}^{255} \big\{ w(z/255) \nabla^2 g(z) \big\}^2. \tag{8}$$

The first term in Equation (8) is the *data term*, and encourages values g and L_{ij} to be such that intensities in linear images would scale linearly with exposure time. As we discussed in class, the weights w have to do with the fact that the linear estimates should rely more on well-exposed pixels than on under-exposed or over-exposed pixels. Review the previously described "Weighting schemes" about what weights exactly you should use. Note that the input to w is divided by 255, because the definitions of the weights assume that the input intensities are in the range $[0, 1]$, whereas the ones you use here are in the range $\{0, \dots, 255\}$.

The second term in Equation (8) is a *regularization term*, and encourages g to be smooth by penalizing solutions g that have large second-derivative magnitudes. Given that g is discrete, the second derivative can be approximated using a Laplacian filter, that is, $\nabla^2 g(z) = g(z+1) - 2g(z) + g(z-1)$. The parameter λ controls how strongly this regularization affects the final result; it is a hyperparameter that you will need to experiment with. Note that, when using the photon-optimal weights w_{photon} that require knowing exposure time, you can set the weights *of the regularization term only* to a constant, $w(z) = 1$.

Solve the *least-squares* optimization problem of Equation (8) by expressing it in matrix form:

$$\|\boldsymbol{A}\boldsymbol{v} - \boldsymbol{b}\|^2, \tag{9}$$

where $\boldsymbol{A}$ is a matrix, $\boldsymbol{v} = [g; \log(L_{ij})]$ are the unknowns, and $\boldsymbol{b}$ is a known vector. Then, use one of `numpy`'s solvers to recover the unknowns. (See `numpy` function `numpy.linalg.lstsq`.)

While Debevec and Malik [1] recover a different g for each color channel, for this homework we recommend that you process pixels from all three channels simultaneously to recover a single g for all channels. This helps reduce color artifacts in the final HDR composite.

Use the function to convert the non-linear images I_{ij}^k into linear ones,

$$I_{ij,\text{lin}}^k = \exp\big(g\big(I_{ij}^k\big)\big). \tag{10}$$

You will not use the values L_{ij} you recovered from solving (8). Include a plot of the function g you recovered. Note that, when creating an HDR image from the `.JPG` stack, you need to use the same weighting scheme in both Eqs (8) (linearization) and (1) (merging).

Develop an HDR image from the `.JPG` stack (5 points). Create an HDR image with and without applying the g correction. compare between both results.

Deliverables

Your submitted solution should include the following:

- A PDF report explaining what you did for each problem, including answers to all questions asked throughout the document. The report should include any figures and intermediate results that you think may help. Make sure to include explanations of any issues that you may have run into that prevented you from fully solving the assignment, as this will help us determine partial credit. The report should also explain any additional image files you include in your solution (see below).
- Your Python code, including code for the bonus problems, and a `README` file explaining how to use the code.
- The HDR images that you create in parts 1 through 4. You can also include additional image files, LDR or HDR, for various experiments (e.g., tonemapping with different values) other than your final ones, if you think they show something important.

Hints and Information

`dcraw` version. Make sure to download and install the latest version of `dcraw`. In particular, the default version that comes in older Windows versions does not support the cameras used in this class, and will produce results with a strong purple hue.

Memory management. When working with the provided and captured exposure stacks, you will notice that your algorithms will be using *a lot of memory*. This is a common issue when processing photographs captured with modern cameras, due to the very large number of pixels these cameras have. At 24 Megapixels, the Nikon D3300 used for this assignment is at the mid-range of megapixels. Still, at this resolution, a 3-channel HDR image takes up more than 0.5 GB of memory. Therefore, feel free to downscale your images to a more manageable size (we suggest a resizing with a factor of 4), especially while you are still debugging your code.

One place in particular where you might run into memory problems is when solving the linear system (9) to recover the non-linear map g. As Debevec and Malik [1] suggest, you should greatly downsample the input images before forming the linear system. However, in this part you should *not* resize the image with `skimage.transform.resize`, or try to blur it before downsampling. For inferring g, all you have to do is downsample an input image `I` with `I[::n, ::n]`, for some n. We recommend using $n = 200$.

Zero weights. When merging many LDR images to HDR ones, you may end up with pixels for which there are not any well-exposed values (i.e., the sum of weights in the denominators of Equation (1) is exactly 0).

You can set those pixels to equal the maximum or minimum valid pixel value of your HDR image, respectively for problematic pixels that are always over-exposed or always under-exposed.

Gamma encoding. Even with tonemapping, your images may appear too dark. In practice, after tonemapping, you still need to apply gamma encoding for images to be displayed correctly. Therefore, you may use further gamma-encoding on your tonemapped or HDR images before displaying them. Gamma encoding will help also when displaying intermediate results (see below).

HDR viewer. If you want to view the `.HDR` files you create, you *cannot* do so with a standard image viewer. Instead, you should use a dedicated viewer for `.HDR` files, such as `OpenHDR`. This viewer provides interactive sliders for controlling exposure (the scaling factor you apply to the image) and gamma encoding, making it easier to find good settings for examining your HDR image.

Showing cropped image regions. In this assignment, you need to select the individual color checker squares multiple times (see `cv2.selectROI`). It is recommended to save these patches or their coordinates to a file to avoid redoing the tedious patch selection stage.

Credits

This assignment was adapted from an assignment originally created by Ioannis Gkioulekas at Carnegie Mellon University. According to Ioannis: "Some inspiration for this assignment and the write-up came from James Hays' and Oliver Cossairt's computational photography courses at Brown University and Northwestern University, respectively. The code for reading ground-truth color checker RGB values is from Ivo Ihrke's color calibration toolbox."

References

[1] P. E. Debevec and J. Malik. Recovering high dynamic range radiance maps from photographs. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '97, pages 369–378, New York, NY, USA, 1997. ACM Press/Addison-Wesley Publishing Co.

[2] S. W. Hasinoff, F. Durand, and W. T. Freeman. Noise-optimal capture for high dynamic range photography. In *Computer Vision and Pattern Recognition (CVPR), 2010 IEEE Conference on*, pages 553–560. IEEE, 2010.

[3] K. Kirk and H. J. Andersen. Noise characterization of weighting schemes for combination of multiple exposures. In *BMVC*, volume 3, pages 1129–1138, 2006.

[4] E. Reinhard, M. Stark, P. Shirley, and J. Ferwerda. Photographic tone reproduction for digital images. In *Proceedings of the 29th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '02, pages 267–276, New York, NY, USA, 2002. ACM.